Aprendizaje por refuerzo profundo con Double DQN en Pong: exploración, valores Q y arquitectura

Daniel Sebastian Velasco Munar[1]

[1]Maestría en Inteligencia Artificial, Pontificia Universidad Javeriana, Bogotá, Colombia
Profesor: Andrés Darío Moreno Barbosa

***Resumen*—Se entrenó un agente Double DQN para jugar Pong de Atari a partir de los píxeles de la pantalla, con un presupuesto de cómputo de una GPU T4. Se corrigió el manejo de los episodios cortados por tiempo y se registraron los valores Q, los objetivos de diferencia temporal (TD) y el error TD. La configuración de referencia se entrenó durante 600 000 pasos, y sobre los primeros 250 000 pasos se compararon dos tasas de decaimiento de ε, una más rápida y otra más lenta que la de referencia, y una red con cabeza Dueling del mismo tamaño. El agente final obtuvo un puntaje medio de +2.4 en cinco partidas de evaluación, frente a −12.4 del agente heurístico que sigue la pelota y −20.0 del agente aleatorio. Bajar ε a 0.02 en 100 000 pasos funcionó mejor que hacerlo en 20 000 pasos o dejarlo en 0.10. La cabeza Dueling aprendió más rápido y sus valores Q quedaron más cerca del retorno real. En todos los agentes el Q predicho superó el retorno descontado observado, lo que indica que Double DQN redujo la sobreestimación sin eliminarla.**

***Declaración de impacto*—Este trabajo muestra qué se puede esperar de Double DQN en Pong con unas cuatro horas de GPU. Con ese presupuesto, la tasa de decaimiento de ε fue la decisión con mayor efecto, y cambiar la estructura de la red aceleró el aprendizaje sin aumentar el número de parámetros. Los resultados también muestran que un error TD estable no garantiza valores Q correctos, por lo que conviene compararlos con el retorno real. Estas observaciones sirven de guía para entrenar agentes de aprendizaje por refuerzo con recursos limitados.**

***Términos de indexación*—Aprendizaje por refuerzo profundo, Atari, Double DQN, exploración ε-greedy, red Dueling, sobreestimación.**

I. Introducción

El aprendizaje por refuerzo estudia cómo un agente aprende a tomar decisiones a partir de las recompensas que recibe del entorno [1]. DQN [2] combinó Q-learning con una red convolucional y alcanzó un nivel cercano al humano en juegos de Atari usando solo los píxeles de la pantalla. Double DQN [3] corrige la tendencia de DQN a sobreestimar los valores Q al separar la elección de la acción de su evaluación.

Este informe documenta el Taller 2: completar, entrenar y evaluar un agente Double DQN en Pong, usando el entorno ALE/Pong-v5 de Gymnasium [4], [5]. Se desarrollan cinco actividades: el ciclo de entrenamiento (Sección II-C y III-A), la estrategia de exploración ε-greedy (III-B), el análisis de los valores Q y del error TD (III-C), la revisión de la arquitectura (II-D y III-D) y la evaluación frente a un agente heurístico (III-E). El código se implementó en TensorFlow y Keras y se ejecutó en Google Colab.

II. Método

A. Entorno y preprocesamiento

En Pong el agente controla la raqueta derecha. Cada punto vale +1 o −1 y la partida termina cuando un jugador llega a 21, así que el puntaje por partida va de −21 a +21. El agente tiene seis acciones. Se usó el preprocesamiento habitual de Mnih *et al.* [2]: imágenes en escala de grises de 84×84, un estado formado por cuatro cuadros consecutivos y un salto de cuatro cuadros por acción. Además, se usó la repetición estocástica de acciones propuesta por Machado *et al.* [6]: en cada paso el emulador ignora la acción elegida y repite la anterior con probabilidad 0.25. Esto vuelve el entorno no determinista y evita que el agente memorice secuencias fijas de acciones.

B. Double DQN

La red principal $Q(s, a; \theta)$ se entrena para acercarse al objetivo

$$y_t = r_t + \gamma(1 - d_t)\hat{Q}(s_{t+1}, a^*; \theta^-) \quad (1)$$

$$a^* = \arg\max_a Q(s_{t+1}, a; \theta) \quad (2)$$

donde $\hat{Q}$ es la red objetivo, con parámetros θ^- copiados de θ cada 1 000 pasos, $\gamma = 0.99$ es el factor de descuento y d_t vale 1 si el episodio terminó. La acción a^* del estado siguiente la escoge la red principal y la red objetivo la evalúa. En DQN ambas operaciones usan $\hat{Q}$, y el máximo sobre estimaciones con ruido tiende a quedar por encima del valor real [3]. La red se actualiza minimizando la pérdida de Huber ℓ_H sobre el error

Informe presentado el 9 de noviembre de 2026 para el curso Aprendizaje Profundo de la Maestría en Inteligencia Artificial, Pontificia Universidad Javeriana, periodo 2026-30.

D. S. Velasco Munar es estudiante de la Maestría en Inteligencia Artificial de la Pontificia Universidad Javeriana, Bogotá, Colombia.

El código fuente, las figuras y las tablas de resultados se entregan junto con este informe.

TD,

$$L(\theta) = \mathrm{E}\left[\ell_{\mathrm{H}}\left(y_t - Q(s_t, a_t; \theta)\right)\right] \quad (3)$$

con lotes de 32 transiciones tomadas al azar de la memoria de experiencias.

C. Ciclo de entrenamiento

El notebook de Jupyter entregado tenía la red, la memoria de experiencias y un paso de entrenamiento, pero no el ciclo: el paso se ejecutaba una sola vez y la red objetivo nunca se sincronizaba. Se implementó el ciclo de entrenamiento de Mnih *et al.* [2], usando como valor objetivo de la red el y_t definido en (1). En cada paso se elige la acción con ε-greedy, se ejecuta y se guarda la transición; cuando la memoria tiene 10 000 transiciones se actualiza θ con un lote, y cada 1 000 pasos se copia θ en θ^{-}.

Se hicieron tres cambios adicionales. Primero, el valor futuro en (1) solo se anula cuando el episodio termina en el juego, no cuando se corta por límite de tiempo, porque en ese caso el estado siguiente sí tiene valor. Segundo, la memoria guarda cada estado y solo el cuadro nuevo del siguiente, ya que comparten tres de sus cuatro cuadros; así cabe el doble de experiencia en la misma memoria. Tercero, la selección de acciones y el paso de entrenamiento se compilaron como grafos de TensorFlow, lo que dio unos 100 pasos por segundo en la GPU.

Los hiperparámetros siguen a [2] con un entrenamiento más corto: Adam con tasa de aprendizaje de 10^{-4}, memoria de 50 000 transiciones, una actualización por paso y ε que baja linealmente de 1.0 a 0.02 en 100 000 pasos. Cada 10 000 pasos se mide el Q máximo medio sobre 500 estados fijos tomados con una política aleatoria, como en [2]; como esos estados no cambian, la curva refleja solo los cambios de la red.

D. Arquitectura

La red del notebook de Jupyter es la propuesta por Mnih *et al.* [2]: tres convoluciones (32 filtros de 8×8 con paso 4, 64 de 4×4 con paso 2 y 64 de 3×3 con paso 1), una capa densa de 512 neuronas con ReLU y una salida lineal con un valor por acción, con 1.69 millones de parámetros en total. Para Pong es adecuada: la escena solo tiene dos raquetas y una pelota, y tres convoluciones bastan para ubicarlas. La salida lineal es correcta porque los valores Q no están acotados, y no se usa normalización por lotes porque los objetivos cambian durante todo el entrenamiento. El punto débil es la capa densa, que concentra el 95 % de los parámetros y obliga a una sola salida a aprender a la vez el valor del estado y la conveniencia de cada acción.

Como modificación se usó la cabeza Dueling [7], que separa el valor del estado $V(s)$ de la ventaja de cada acción $A(s, a)$ y los combina como

$$Q(s,a) = V(s) + A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a') \quad (4)$$

En Pong la acción casi no importa en muchos estados, por ejemplo cuando la pelota se aleja hacia el rival; con esta cabeza la red puede aprender $V(s)$ en esos estados sin ajustar cada Q por separado. Las convoluciones no cambian y las dos ramas de 256 neuronas dejan 1.686 millones de parámetros, casi los mismos de la red original, de modo que la comparación mide el efecto de la estructura y no el del tamaño.

E. Diseño experimental

Un paso es una decisión del agente, equivalente a cuatro cuadros del juego. Por el tiempo de cómputo disponible, la corrida base se entrenó 600 000 pasos y las demás 250 000 (Tabla I). Todas las comparaciones se hacen sobre los primeros 250 000 pasos, con la misma semilla y los mismos pesos iniciales, de modo que la única diferencia sea el cambio indicado.

TABLA I
CONFIGURACIONES ENTRENADAS

Corrida	Cambio frente a base	Pasos
base	red original; ε de 1.0 a 0.02 en 100 000 pasos	600 000
eps_rapido	ε de 1.0 a 0.02 en 20 000 pasos	250 000
eps_lento	ε de 1.0 a 0.10 en 250 000 pasos	250 000
dueling	cabeza Dueling de igual tamaño	250 000

En la evaluación cada agente jugó 5 partidas completas con las mismas semillas, y los agentes DQN usaron ε = 0.01. Se compararon con un agente aleatorio y con el agente heurístico del notebook de Jupyter, que mueve la raqueta hacia la altura de la pelota y juega con el mismo salto de cuadros y la misma repetición estocástica de acciones. En los agentes DQN también se registró el Q medio de las acciones elegidas y el retorno descontado real desde cada estado visitado: si el primero queda por encima del segundo, hay sobreestimación [3].

III. RESULTADOS Y ANÁLISIS

A. Corrida base

Durante los primeros 60 000 pasos la recompensa se mantuvo cerca de −20, lo esperado mientras ε es alto y casi todas las acciones son al azar (Fig. 1). La mejora empezó cuando ε bajó de 0.5 y fue sostenida entre los 100 000 y los 270 000 pasos: la media móvil pasó de −17 a −3 y superó −15 por primera vez en el paso 134 780. Entre los 270 000 y los 500 000 pasos la curva se estabilizó entre −7 y −2, es decir, el agente perdía por pocos puntos.

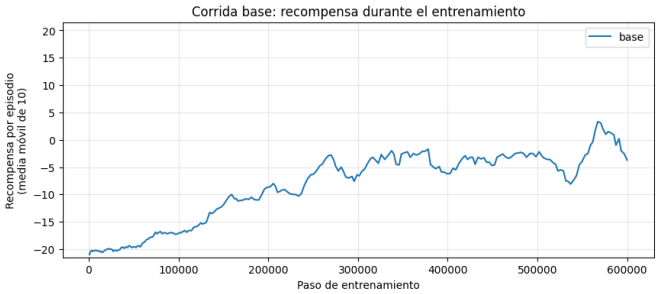


Fig. 1. Recompensa por episodio de la corrida base durante 600 000 pasos.

Cerca del paso 530 000 hubo una caída hasta −8, seguida del mejor tramo de la corrida (media móvil de +3.3 alrededor del paso 565 000). Estas subidas y caídas, sin ningún cambio en la configuración, son comunes en DQN: al cambiar la política cambian los datos que llegan a la memoria, y el objetivo se mueve con cada sincronización de la red objetivo. Con 600 000

pasos el agente aprendió a devolver la pelota y a ganar algunas partidas, aunque el entrenamiento fue unas 20 veces más corto que el de Mnih *et al.* [2].

B. *Estrategia de exploración*

La tasa de decaimiento de ε de la corrida base fue la mejor de las tres (Fig. 2 y Tabla II). A los 250 000 pasos su media en los últimos 10 episodios fue −6.4, frente a −12.4 de eps_rapido y −16.0 de eps_lento, y fue el primero en superar una media de −15.

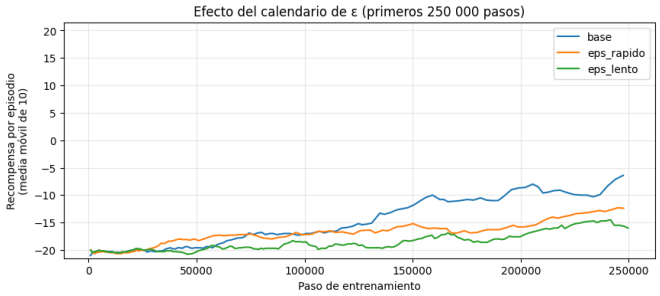


Fig. 2. Recompensa durante los primeros 250 000 pasos con las tres tasas de decaimiento de ε.

eps_rapido mejoró antes que los demás entre los 30 000 y los 100 000 pasos, cuando base todavía exploraba mucho, pero después avanzó despacio y base lo superó cerca del paso 110 000. Al dejar de explorar tan pronto, la memoria se llenó con experiencia de una política todavía mala y con poca variedad de situaciones.

eps_lento se mantuvo cerca de −19 hasta los 150 000 pasos. Parte de la diferencia se debe a que nunca baja de 10 % de acciones al azar, pero la evaluación con ε = 0.01 (Tabla III) muestra que también aprendió menos: obtuvo −13.6, frente a −5.0 de base y −10.4 de eps_rapido. Además jugó 187 episodios contra 132 de base en el mismo número de pasos, es decir, perdía más rápido. El resultado refleja el compromiso entre explorar y explotar [1]: explorar muy poco deja experiencias pobres y explorar demasiado retrasa el uso de lo aprendido.

TABLA II
RESULTADOS DE ENTRENAMIENTO

Corrida	Episodios	Media últimos 10	Mejor media móvil	Paso con media ≥ −15
base 600k	246	−3.7	3.3	134 780
base 250k	132	−6.4	−6.4	134 780
eps_rapido	118	−12.4	−12.3	209 850
eps_lento	187	−16.0	−14.5	229 280
dueling	137	−5.2	−1.8	113 315

C. *Valores Q y error TD*

La Fig. 3 resume la corrida base. El Q predicho y el objetivo de (1) quedaron casi superpuestos durante toda la corrida, porque el objetivo se calcula con una copia de la red de hace menos de 1 000 pasos. Por eso esta comparación no permite detectar la sobreestimación; para eso se usa el retorno real (Sección III-E).

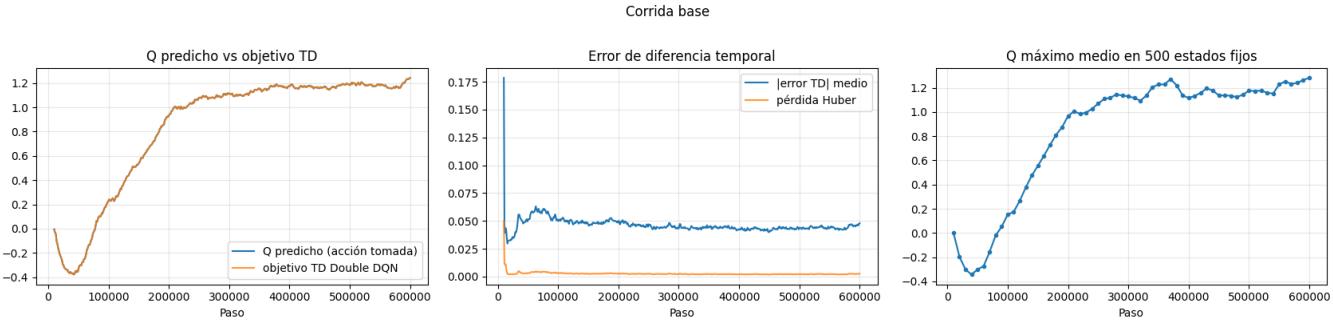


Fig. 3. Corrida base. Izquierda: Q predicho de la acción tomada frente al objetivo Double DQN. Centro: |error TD| medio y pérdida de Huber. Derecha: Q máximo medio en 500 estados fijos.

En los primeros 40 000 pasos Q bajó hasta cerca de −0.35, porque el agente casi solo perdía puntos. Luego subió junto con la recompensa hasta cerca de 1.0 en el paso 200 000 y se mantuvo entre 1.1 y 1.3 hasta el final, sin crecer sin control, así que no hubo divergencia. El Q máximo en los estados fijos tuvo la misma forma, lo que confirma que el cambio viene de la red y no de los estados visitados.

Después de un pico inicial de 0.18, el |error TD| medio se mantuvo entre 0.04 y 0.06 y la pérdida de Huber cerca de 0.003. No bajó a cero porque el objetivo se mueve, pero tampoco creció, lo que indica un entrenamiento estable.

La estabilidad no implica que los valores sean correctos. Un Q cercano a 1.2 indicaría que el agente espera ganar puntos con frecuencia, pero en la evaluación el retorno descontado real fue 0.08 para base a 600 000 pasos, mientras el Q medio fue 1.34. A los 250 000 pasos la diferencia fue parecida (1.10 frente a −0.16): el agente perdía las partidas y la red seguía prediciendo valores positivos. Double DQN reduce la sobreestimación de DQN [3], pero en este experimento no la eliminó.

D. *Arquitectura*

La cabeza Dueling aprendió más rápido (Fig. 4). Las dos curvas fueron casi iguales hasta el paso 60 000, cuando ε todavía era alto; desde ahí Dueling fue adelante, superó una media de −15 en el paso 113 315 (frente a 134 780) y alcanzó su mejor media móvil, −1.8, cerca del paso 195 000, cuando la red original estaba en −9. Después del paso 210 000 bajó hasta −7 y terminó en −5.2, cerca de la original (−6.4), así que la ventaja fue sobre todo en velocidad.

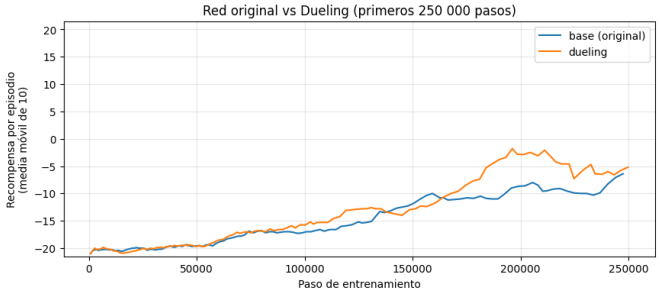


Fig. 4. Red original frente a la red Dueling en los primeros 250 000 pasos.

En la evaluación Dueling obtuvo −1.8 frente a −5.0 de la red original, pero con 5 partidas y desviaciones de 5.4 y 4.5 puntos esa diferencia no es concluyente. Lo más claro se vio en los valores Q: los de Dueling fueron más bajos (0.83 frente a 1.10) y más cercanos al retorno real (diferencia de 0.88 frente a 1.26), con un error TD igual de estable. Esto concuerda con la motivación de la arquitectura, aunque con una sola semilla es una indicación y no una prueba.

E. Evaluación de la política

La Tabla III y la Fig. 5 muestran la evaluación. El agente aleatorio perdió todas las partidas por 19 a 21 puntos. El heurístico obtuvo −12.4; sus partidas fueron las más largas (unos 4 700 pasos) porque devuelve muchas pelotas, pero pierde puntos cuando la pelota cambia de dirección rápido, probablemente por el retraso que agregan el salto de cuadros y la repetición estocástica de acciones.

TABLA III
EVALUACIÓN FINAL DE LOS AGENTES

Agente	Puntaje medio	Desv.	Q medio	Retorno real
aleatorio	−20.0	0.71	–	–
heurístico	−12.4	3.97	–	–
base 250k	−5.0	4.53	1.10	−0.16
eps_rapido 250k	−10.4	2.30	0.64	−0.34
eps_lento 250k	−13.6	3.05	1.58	−0.54
dueling 250k	−1.8	5.36	0.83	−0.05
base 600k	+2.4	4.83	1.34	0.08

Todos los agentes Double DQN superaron al aleatorio, y todos menos eps_lento superaron al heurístico. Base a 600 000 pasos fue el único con media positiva (+2.4): ganó 4 de las 5 partidas por entre 3 y 6 puntos y perdió una por 6. Superó al heurístico por casi 15 puntos, y la mejora de 7.4 puntos frente a sí mismo a los 250 000 pasos muestra que el entrenamiento adicional sí sirvió.

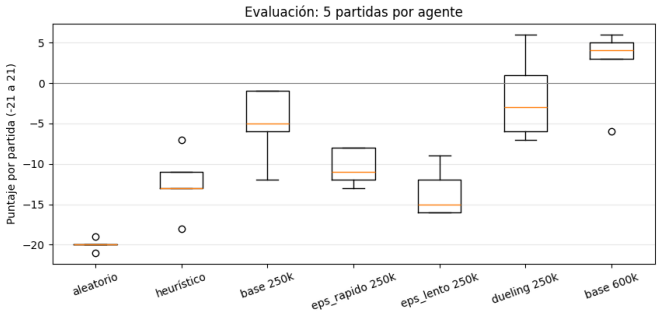


Fig. 5. Puntaje por partida de cada agente en la evaluación.

Con 5 partidas por agente las desviaciones fueron de 2 a 5 puntos, así que solo las diferencias grandes (aleatorio frente a DQN, heurístico frente a base 600k) son confiables. En todos los agentes el Q medio quedó por encima del retorno real, y la mayor diferencia se dio en eps_lento (1.58 frente a −0.54), el agente que peor jugó.

IV. CONCLUSIONES

Se completó el ciclo de entrenamiento de Double DQN y se corrigió el manejo de los episodios cortados por tiempo. Con 600 000 pasos, unos 100 minutos en una GPU T4, el agente pasó de perder 21 a 0 a ganar 4 de 5 partidas de evaluación, con un puntaje medio de +2.4 frente a −12.4 del heurístico que sigue la pelota.

Con este presupuesto, la tasa de decaimiento de ε tuvo un efecto grande: bajar ε a 0.02 en 100 000 pasos funcionó mejor que hacerlo en 20 000 pasos o dejarlo en 0.10, tanto en entrenamiento como en evaluación. La cabeza Dueling, con casi los mismos parámetros que la red original, aprendió más rápido y sus valores Q quedaron más cerca del retorno real, aunque al final de los 250 000 pasos la ventaja en puntaje fue pequeña.

Los valores Q fueron estables, pero quedaron por encima del retorno real en todos los agentes: Double DQN redujo la sobreestimación sin eliminarla, y un error TD estable no basta para confiar en la escala de Q.

Las conclusiones tienen tres limitaciones: cada configuración se entrenó con una sola semilla, la evaluación usa 5 partidas y el entrenamiento fue unas 20 veces más corto que el de Mnih *et al.* [2]. Con más pasos, varias semillas y mejoras como la repetición priorizada de experiencias [8], se esperaría un agente más fuerte y resultados más firmes.

REFERENCIAS

[1] R. S. Sutton y A. G. Barto, *Reinforcement Learning: An Introduction*, 2.ª ed. Cambridge, MA, EE. UU.: MIT Press, 2018.

[2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, feb. 2015.

[3] H. van Hasselt, A. Guez y D. Silver, "Deep reinforcement learning with double Q-learning," en *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.

[4] M. G. Bellemare, Y. Naddaf, J. Veness y M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, 2013.

[5] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[6] M. C. Machado *et al.*, "Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents," *J. Artif. Intell. Res.*, vol. 61, pp. 523–562, 2018.

[7] Z. Wang *et al.*, "Dueling network architectures for deep reinforcement learning," en *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.

[8] T. Schaul, J. Quan, I. Antonoglou y D. Silver, "Prioritized experience replay," en *Proc. Int. Conf. Learn. Represent.*, 2016.